

Variables

Variables are a core concept in coding, and nearly all languages utilize them. We can think of a variable as a container that holds data, which we can then reference anywhere in our code. This can be a very powerful tool whenever we want to use the same information in multiple places, or if we need to perform a series of calculations and store their results.

Declaring a Variable

Whenever we create a variable we call it "declaring". Typically we will declare variables at the top of a script, above the `ready()` function and below the "extends" keyword. The declaration of a variable in GDScript has three parts: the "var" keyword, a name for the variable, and an optional value assignment. Here's an example below:

```
var my_variable_name = 5
```

It's important to mention that variable names are case sensitive, meaning that if we have two variables and one starts with a capital letter and the other a lower case, even if they are the same word, there will be two different variables created! Here's an example:

```
# Our first variable
var my_variable = 1

# A new variable
var My_variable = 2
```

For this reason, it's important to be consistent when naming variables. It is best to follow these guidelines:

- Variable names should describe the data they store or what it will be used for
- Keep your names short, you may be writing them often
- Only use lowercase letters when declaring a variable

More ways of declaring variables

Some times we might have specific use cases for a variable, or we might want to be able to more easily edit them. Below are some helpful tags and variable declarations:

If we want to add a variable to the inspector so that we may edit its value easier, we can use the `@export` tag when we declare it:

```
# This variable will appear in the inspector when we click on the node the
script is attached to.
@export var my_exported_variable = 5
```

If we want to make a variable whose value will never change, we can use the "const" keyword:

```
const my_constant_variable = 5
```

If we need a variable to wait until the scene is ready to become active we can use the @onready tag. (This can be important if a reference to another node is stored in the variable):

```
@onready var my_onready_variable = 5
```

Storing Data in a Variable (Assignment)

We can think of a variable as a box that can hold one piece of data at a time, and we call the act of storing data in a variable *assignment*. So far we've talked about assigning a default value to a variable, but we can assign a new value to a variable in any function using the = sign. Whenever we assign a new piece of data to a variable the old data is removed (and lost forever!). This means that we must be careful when assigning new data to a variable, as we do not want to accidentally erase data that we may still need or put data in a variable that won't work with the kinds of calculations we are using. Below is an example of a *declaration* and an *assignment*:

```
# This is a declaration
var my_new_variable = 6

func _ready():
    # Here we assign the data "Hello World!" to the variable, the previous
    data, 6,
    # is then removed and inaccessible.
    my_new_variable = "Hello World!"
```

We can also declare a variable and then assign a value later. If we do not assign a value to a variable it will hold the value 'nil' until we assign a value.

```
func _process():
    var local_process_variable
    local_process_variable = 12
```

Variable Scope

Above we said that we *usually* declare variables at the top of a script. This has to do with the variable's **scope**, or where it can be *called* (this is what we call it when we use a variable) in our script. A variable declared at the 0-indent level can be called anywhere within our script, we may refer to this kind of variable as a "script-wide" variable. Here's an example:

```
extends Node2D

var my_script_wide_variable = "Hello World!"

func _ready():
    # The my_script_wide_variable variable can be used in this function
    my_script_wide_variable
    pass

func _process(delta):
    # The my_script_wide_variable variable can be used in this function as
    well
    my_script_wide_variable
    pass
```

However, some times we only need a variable to be used in one function. A variable that is declared at the 1-indent level or above is referred to as a "local" variable. These variables are only accessible within the function or code block they are declared in, and cannot be accessed at a lower indent level. Below are some examples

```
func _ready():
    # This variable can only be used within the ready function
    var ready_variable = 6

func _process(delta):
    # This variable can only be used within the process function
    var process_variable = 9

func my_func():
    # This variable can only be used within the my_func function
    var my_func_variable = 10

    if true:
        # This variable can only be used inside the "true" section of this
        # conditional. It cannot be used in other parts of the my_func
```

```
function.  
    var my_conditional_variable
```

Now it's your turn!

Below are a examples of variable declarations, label them as either "local" or "script-wide" variables

```
@export var speed = 400
```

```
var player = 2
```

```
func _ready():  
    var starting_position = 12
```

```
if Input.is_action_pressed('ui_accept'):  
    var jump = true
```

```
const max_health = 100
```

Next, write the following declarations in a default Node2D script:

- A local variable that can only be used in the ready() function with no default value
- A script-wide variable for the max speed of the player that cannot be changed
- A script-wide variable for the player's jump power that can be edited in the inspector
- A script-wide variable with the default value 12
- A local variable that can only be used in the process() function with the default value 7

And finally, assign the following values in the following places:

- Assign the value "Hello World!" to the local ready() function variable *not* in its declaration (you should assign the value on a new line)
- In the ready() function, assign a new value to the jump power variable that is two times the value currently stored in it. Do not hard code this (don't write a number).
- In the process function, add the "default value 12 variable" and the local process variable together and then assign that value to the "default value 12" variable